

Pratikar

Technology Stack and Architecture

Version 1.0 · **Date** 17 September 2026 · **Status** MVP in development, Grand Final 20 September 2026
Team Sid · Pratham · Navya · Parnika · Dhiraj · **Companion documents** Pratikar PRD, Pratikar BRD

This document specifies how Pratikar is built. The PRD defines what it does; the BRD defines why it should exist.

Source tags used throughout. **Deck** stated in the finalized submission deck **Rec** our technical recommendation, not previously specified **TBD** requires a team decision before build.

1. Technology Overview

Pratikar is a stateless document-analysis web application. A user uploads two documents; the system extracts structured facts, evaluates them along two independent reasoning paths, merges the results into a cited verdict, and generates an appeal document. Nothing persists beyond the session unless the user saves it.

Three properties drive every technology choice below:

- No gated dependencies.** No licence, regulator approval or corporate entity is required for any component. Every input is either supplied by the user or publicly published. **Deck**
- Grounding is architectural, not prompted.** A deterministic rule engine runs before any model call, and generated output is constrained to text present in the user's own documents. This shapes the component boundaries, not just the prompts. **Deck**
- Two developers, three days remaining.** One language on the server, one hosting provider per tier, free tiers throughout, and no component that requires specialist operational knowledge. **Rec**

2. Recommended Technology Stack

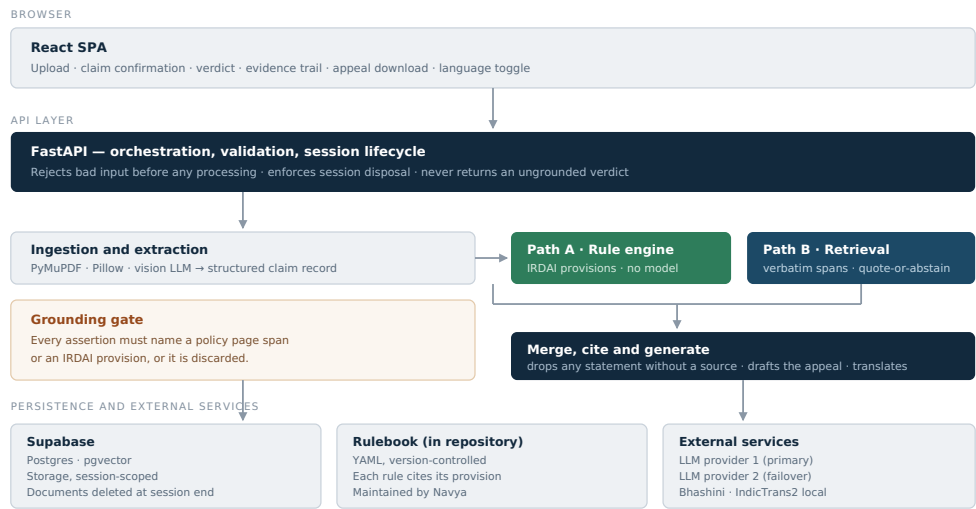
Layer	Technology	Purpose	Reason for selection	Alternative
Frontend	React + Vite + Tailwind Deck	Upload, verdict, evidence trail and appeal screens	Fastest path to a polished demo; Parnika can build components against Pratham's scaffolding without deep JavaScript	Next.js — rejected, SSR adds no value to a single-session tool
Backend	FastAPI, Python 3.11 Deck	API surface, orchestration, rule engine host	Document processing is Python-native. One server language avoids splitting two developers across stacks	Node + Express — would force the PDF work into a weaker ecosystem
PDF parsing	PyMuPDF Deck	Extract page-scoped text from policy wordings	Preserves page numbers and character offsets, which the evidence trail depends on	pdfplumber — slower on 80-page documents
Image handling	Pillow Rec	Deskew and normalise photographed letters before extraction	Improves vision extraction on phone photos at negligible cost	OpenCV — heavier than needed
Vision extraction	Vision-capable LLM Deck	Read a photographed rejection letter into text	Handles varied insurer letter formats without per-insurer templates	Tesseract OCR — poor on varied layouts
Reasoning	Long-context LLM, second provider configured as fallback Deck	Field extraction, clause reasoning, appeal drafting	A 40-80 page policy fits in context. Two providers means no single point of failure on demo day	Single provider — rejected on demo-day risk

Layer	Technology	Purpose	Reason for selection	Alternative
Embeddings	Provider embedding API, local sentence-transformers as fallback Rec	Locate candidate clauses within the policy	Retrieval narrows the context; the LLM still quotes verbatim from the retrieved span	Keyword search alone — misses paraphrased clause references
Vector store	pgvector on Supabase Deck	Store and query page-scoped policy chunks	Postgres, vectors, storage and auth on one free tier — one service to operate, not four	ChromaDB local — acceptable offline fallback
Rule engine	Hand-coded IRDAI provisions as YAML, evaluated in Python Deck	Deterministic regulatory checks	Rules are data, so Navya can read and extend them without touching application code	Hard-coded conditionals — would make the rulebook developer-only
Translation	Bhashini API, IndicTrans2 offline fallback Deck	Hindi output for verdict and appeal letter	Free, government-backed, built for Indian languages. Offline path removes demo-day network risk	General-purpose LLM translation — weaker on Indic scripts
Document generation	ReportLab with Noto Sans Devanagari Rec	Render the appeal letter as a downloadable PDF	Devanagari requires a font with those glyphs; the default PDF fonts do not have them	python-docx — .docx output, no font embedding control
Database	Supabase Postgres Deck	Session records, extracted claim data, rule results	Comes with the vector store already chosen; no second service	SQLite — no managed hosting path
Object storage	Supabase Storage Rec	Uploaded documents during a session only	Same provider, same credentials, lifecycle deletion supported	S3 — extra account and credentials for no gain
Frontend hosting	Vercel free tier Deck	Serve the web application	Deploy from git in minutes; no configuration to get wrong under time pressure	Netlify — equivalent
Backend hosting	Render free tier Deck	Host the API	Free tier sufficient for a single-session demo. Cold starts mitigated by warming before the demo	Railway — equivalent
Version control / CI	GitHub + GitHub Actions Rec	Source control, test run on push	Runs Dhiraj's ground-truth suite automatically so regressions surface the same day	Manual testing — unreliable under deadline
Observability	Structured JSON logging to stdout Rec	Stage timings, rules fired, grounding violations	Sufficient to answer "how do you know it worked" with evidence. No document content ever logged	Sentry — useful later, unnecessary overhead now

Deliberately not in the stack. Kubernetes, message queues, Redis, a separate auth provider, a feature-flag service, a model-serving framework, and any fine-tuned or self-trained model. None is required by the MVP, and each would consume time the team does not have. [Rec](#)

3. System Architecture

Two reasoning paths run independently from the same structured input and are merged only at the end. Neither can produce an answer the other cannot support.



The load-bearing decision. Path A is deterministic and its results are binding. Where a coded provision settles the question, the verdict follows the rule and Path B supplies supporting quotation rather than judgement. This is why the answer to "what if the AI is wrong" is a diagram rather than a reassurance. [Deck](#)

4. Component Architecture

Component	Responsibility	Input	Output	Dependencies	Owner
Upload handler	Validate type, size and content; store for the session; reject unsupported documents before any processing	Two files from the browser	Document records with storage references	Supabase Storage	Pratham
Policy parser	Split the policy into page-scoped chunks with page numbers and character offsets preserved	Policy PDF	Indexed chunks with embeddings	PyMuPDF, embeddings, pgvector	Pratham
Letter extractor	Read the rejection letter and produce the structured claim record; return null for absent fields, never a guess	Letter PDF or image	Structured claim record	Pillow, vision LLM	Sid
Rule engine	Evaluate coded IRDAI provisions against the claim record; fully deterministic	Claim record, rulebook	Pass / fail / not-applicable per rule, each citing its provision	Rulebook YAML only	Sid, rules by Navya
Clause retriever	Locate the cited clause and its dependent clauses; return verbatim text with page references	Claim record, indexed chunks	Quoted spans with page numbers	pgvector, LLM	Sid

Component	Responsibility	Input	Output	Dependencies	Owner
Merge and grounding gate	Combine both paths into a verdict; discard any statement lacking a source	Rule results, quoted spans	Verdict, ordered reasons, evidence items	LLM	Sid
Appeal generator	Draft the grievance-officer letter from established grounds only; introduce no new ground	Verdict and evidence	Appeal text, then a PDF	LLM, ReportLab	Sid
Translator	Render verdict and letter in Hindi without changing substance	English text	Hindi text	Bhashini, IndicTrans2	Pratham
Evidence resolver	Given a citation, return the underlying policy text or provision so the user can check it	Evidence reference	Source text and page number	Postgres	Pratham
Session disposer	Delete uploaded documents at session end unless the user saved them	Session identifier	Confirmation	Supabase	Pratham

5. Data Architecture

Data flows in one direction. Nothing written downstream is fed back into extraction, so an error cannot propagate into an apparently-corroborated conclusion.

Stage	Processing	Data produced
Ingestion	Type, size and content validation; virus-safe handling; store under a session key	Document records: filename, MIME type, size, storage path, session identifier
Preprocessing	Policy split into page-scoped chunks, page numbers and character offsets preserved. Photographed letters deskewed and normalised	Policy chunks with page and offset metadata; a normalised letter image
Vectorisation	Each policy chunk embedded and written to pgvector with its page reference attached	Chunk embeddings, page-referenced
Extraction	Vision and text models read the letter into the structured claim record; absent fields return null	Claim record (PRD §12)
Rule evaluation	Deterministic evaluation against the rulebook; no model involved; reproducible on identical input	Rule results with provision references
Retrieval	Vector search narrows candidates; the cited clause and its dependent clauses are quoted verbatim	Evidence spans: quoted text, page number, chunk identifier
Merge	Verdict assembled; statements lacking a source discarded at this boundary	Verdict, ordered reasons, evidence items
Generation	Appeal drafted from established grounds; rendered to PDF; translated on request	Generated document records
Disposal	Uploaded documents and derived chunks deleted at session end unless saved	Nothing retained

Page references are the spine of the system. If a page number is lost anywhere between parsing and output, the evidence trail breaks and the product's central claim fails. This is the one invariant to protect in every refactor.

Rec

6. AI/ML Architecture

No model is trained. There is no training set, no fine-tuning and no custom model. The system orchestrates existing models under hard constraints. [Deck](#)

Model selection

Task	Model class	Why
Letter reading	Vision-capable LLM	Insurer letter formats vary widely; a template-based OCR approach would need per-insurer work the team cannot do in the time available
Field extraction	Long-context LLM, structured output	Schema-constrained output makes a missing field explicit rather than invented
Clause reasoning	Long-context LLM	A 40–80 page policy plus the rule results fit in context, so the model reasons over supplied text rather than recalled knowledge
Appeal drafting	Long-context LLM	Composition from established grounds is a genuine language task
Translation	Bhashini, IndicTrans2	Purpose-built for Indian languages; free; offline fallback removes demo-day network dependency

Prompt architecture

1. **Separation of concerns.** Extraction and reasoning are separate calls. Extraction returns fields or null and is never asked to interpret them.
2. **Context, not recall.** Every reasoning call receives the retrieved policy spans and the rule-engine results. The model is not asked to remember insurance law.
3. **Source-required schema.** Output schema requires a page span identifier or a provision reference per assertion. An assertion without one is invalid output, not low-confidence output.
4. **Explicit abstention.** "Not determinable from the documents provided" is a first-class schema value, not a fallback phrase in the prose.
5. **Untrusted document text.** Policy and letter content is passed as quoted evidence, never as instruction. A document containing instruction-like text must not influence behaviour.

Evaluation and validation

Check	Method	Pass condition
Grounding violations	Automated check on every analysis: every statement resolves to a source	Zero. The only non-negotiable target
Field extraction accuracy	Ground-truth set, run by Dhiraj on every build	Reported with sample size; no threshold claimed
Clause retrieval correctness	Manual verification that quoted text matches the PDF exactly	Reported with sample size
Verdict agreement	Comparison against labelled outcomes	Reported with sample size
Abstention correctness	Deliberate negative cases with no supporting evidence in the documents	All abstain; none produce a verdict
Rule determinism	Same input run repeatedly	Identical rule results every time

Confidence and fallback

No numeric confidence score is displayed. The verdict is a three-level judgement. A percentage attached to a legal conclusion would imply a calibration the system does not have. Moderate exists to carry genuine uncertainty. [Rec](#)

Fallback chain. A failed, timed-out or schema-invalid model call is retried once, then fails over to the second provider. If both fail, the system returns the deterministic rule results alone and states that the interpretive analysis is unavailable. It never returns a verdict assembled from a partial pipeline.

7. API Architecture

Method	Endpoint	Purpose	Input	Output
POST	/api/analyses	Start an analysis from the two documents	Multipart: rejection letter, policy wording	Analysis id, status processing
GET	/api/analyses/{id}	Poll status, then retrieve the verdict	Analysis id	Status; when complete: claim record, verdict, reasons, evidence items
GET	/api/analyses/{id}/evidence/{ref}	Resolve a citation so the user can check it	Evidence reference	Quoted source text, page number, or the provision text
POST	/api/analyses/{id}/appeal	Generate the appeal document	Language: en or hi	Generated document id
GET	/api/analyses/{id}/appeal/{docId}	Download the appeal as PDF	Document id	PDF stream
DELETE	/api/analyses/{id}	Discard the session and its documents	Analysis id	Confirmation
GET	/api/health	Liveness and provider reachability, for demo-day warming	—	Service and provider status

Seven endpoints. No endpoint exists for a feature outside the MVP, and none should be added without a corresponding functional requirement in the PRD.

8. Database Design

Entity	Key fields	Relationships
analysis	id, created_at, status, language, expires_at	Root entity. One per session
document	id, analysis_id, role (letter policy), mime_type, size, storage_path, page_count	Many to one with analysis
claim_record	id, analysis_id, insurer_name, policy_number, claim_reference, claim_amount, rejection_date, stated_ground, cited_clause_ref, policy_inception_date, continuous_months	One to one with analysis
policy_chunk	id, document_id, page_number, char_start, char_end, text, embedding	Many to one with document. Indexed for vector search
rule_result	id, analysis_id, rule_id, provision_ref, outcome (pass fail not_applicable), explanation	Many to one with analysis
verdict	id, analysis_id, level (strong moderate weak), summary, generated_at	One to one with analysis
evidence_item	id, verdict_id, statement, source_type (policy_span provision), policy_chunk_id, provision_ref, ordinal	Many to one with verdict. The evidence trail
generated_document	id, analysis_id, kind (gro_letter grounds_request), language, storage_path	Many to one with analysis

Retention. Every table above is scoped to `analysis`, so deleting one analysis removes everything derived from it. Retention is therefore a single cascade, not a cleanup routine that can drift out of correctness. **Rec**

9. Security Architecture

Area	Approach
Authentication	None in the MVP. There are no accounts, so there are no credentials to steal. A session is identified by an unguessable analysis id held by the browser
Authorization	Possession of the analysis id grants access to that analysis only. Ids are random and unguessable, never sequential
Transport	HTTPS throughout. No document is transmitted over plain HTTP
API security	CORS restricted to the deployed frontend origin. Request size limits enforced before the body is read. Rate limiting at the platform default
Input validation	Type, size and content checked before processing. A file that claims to be a PDF but is not is rejected, not parsed
Secrets	Provider keys in environment variables only. Never in client code, never committed. Model calls are made server-side, so no key ever reaches the browser
Data protection	Documents discarded at session end. No training use; provider settings configured accordingly. No sharing beyond the model call required to process them
Logging	Stage timings, rules fired and grounding violations only. Never document content, never extracted personal data
AI-specific	Prompt injection: policy and letter text is data, never instruction. Passed as quoted evidence, with output constrained by schema. A document containing instruction-like text cannot change system behaviour
Generated documents	Served to the requesting session only. No public URLs, no guessable paths

Honest scope statement. This is a hackathon security posture, not a production one. No penetration testing, no formal threat model, no data-processing agreement, no compliance certification. The strongest protection the MVP has is that it retains almost nothing. Operating this for real users requires all of the above. **Rec**

10. Deployment Architecture

Environment	Hosting	Purpose	Configuration
Development	Local machines	Feature work; fast iteration	<code>.env.local</code> , never committed. Local Postgres or a Supabase dev project
Testing	GitHub Actions	Run Dhiraj's ground-truth suite on every push	Model calls mocked for deterministic tests; rule-engine tests run for real
Production	Vercel (web) + Render (API) + Supabase (data)	The URL used for the mentoring round and the Grand Final	Environment variables set in each platform's dashboard. Deploy from <code>main</code>

Environment variables

`LLM_PRIMARY_KEY` · `LLM_FALLBACK_KEY` · `SUPABASE_URL` · `SUPABASE_SERVICE_KEY` · `BHASHINI_KEY` · `ALLOWED_ORIGIN` · `SESSION_TTL_MINUTES` · `MAX_UPLOAD_MB`

Demo-day operational procedure

1. Deploy the final build on 19 September. No deployment on 20 September under any circumstances.
2. Warm the API before the session so cold starts do not appear as latency in front of judges.
3. Three demo cases pre-cached and returning in under five seconds.
4. Backup recording of the full flow on a local drive, not in the cloud.

5. One unseen letter held in reserve for a judge who asks whether the demo is hardcoded.

11. Development Environment

Area	Choice
Languages	Python 3.11 (backend, rule engine, document processing) · TypeScript (frontend)
Backend frameworks	FastAPI, Pydantic for schema validation, Uvicorn
Key libraries	PyMuPDF, Pillow, ReportLab, PyYAML, supabase-py, httpx
Frontend	React 18, Vite, Tailwind CSS, TanStack Query for polling
Package management	<code>uv</code> or <code>pip</code> with a pinned <code>requirements.txt</code> · <code>npm</code> with a committed lockfile
Version control	Git on GitHub. Short-lived branches, small pull requests, <code>main</code> always deployable
Editor	VS Code with Python and Tailwind extensions. Shared formatter settings committed to the repository
Formatting and linting	<code>ruff</code> for Python, <code>prettier</code> for the frontend. Configured once, applied automatically
Testing	<code>pytest</code> . The rule engine is unit tested for real; model calls are mocked

One convention that matters more than the rest. Pin every dependency version. A transitive upgrade breaking the build on 19 September is an avoidable way to lose the hackathon. **Rec**

12. Team-wise Technical Responsibility

Two members build the pipeline; three own work that is on the critical path for credibility but never on the critical path for code. No first-year member can be blocked by, or block, a merge.

Member	Primary responsibility	Secondary responsibility	Deliverables
Sid	Architecture; rule engine; merge and grounding gate; appeal generation	Integration; driving the live demo	Grounding contract enforced at generation · rule engine evaluating the rulebook · cited verdicts on the demo cases · appeal generator
Pratham	Ingestion, parsing and indexing; API surface; database; deployment	Frontend scaffolding for Parnika; translation integration	Deployed URL from 18 Sep · upload to indexed-chunks pipeline · all seven endpoints · evidence resolver · session disposal
Navya	IRDAI rulebook as YAML, each rule citing its provision; real rejection letter collection	Domain questions during mentoring and judging	Coded rule set with provision references · collected real letters · resolution of the 2020 circular open item
Parnika	Product design; evidence-trail interface; Hindi copy review	Frontend components against Pratham's scaffolding	Three screens designed and implemented · evidence trail as the visual centrepiece · Devanagari rendering verified in the download path
Dhiraj	Ground-truth test set; running it against every build; defect log	Documentation; demo backup recording; mentor and judge Q&A sheet	Labelled case set · test results with sample sizes · recorded demo run · question-and-answer preparation for the team

Why this split works. The rulebook, the test set and the design are files, not code branches. If a first-year member is ever blocked waiting on a pull request, the structure has failed and should be corrected the same day. [Deck](#)

13. MVP Technical Scope

MUST IMPLEMENT

- Dual document upload with validation
- Policy parsing with page references preserved
- Letter extraction into the claim record
- Deterministic IRDAI rule engine
- Verbatim clause retrieval
- Merge with the grounding gate enforced
- Verdict with evidence trail
- One generated appeal document
- Session disposal of uploaded documents
- Second provider configured as failover

SHOULD IMPLEMENT

- Hindi output for verdict and letter
- Flow C: request-for-grounds letter
- Non-advice notice and deadline display
- Demo-case response caching
- Structured logging of stage timings
- Deskewing of photographed letters

DO NOT IMPLEMENT

- Accounts, login, saved history
- Any dashboard or analytics view
- Motor, life or travel rulebooks
- Mobile application
- Insurer or TPA portal
- Automated filing to any portal
- Payments or subscriptions
- Case-precedent search
- Model fine-tuning of any kind
- Languages beyond Hindi

The DO NOT column is a commitment, not a preference. Anything a mentor or judge requests from it becomes a roadmap slide rather than a task. [Deck](#)

14. Technical Development Roadmap

Timeline correction. The submission deck plans 14–16 September for the core pipeline and 17 September for the mentoring round. Today is 17 September. The remaining build window is therefore **three days, not six**, and the plan below is rewritten against the time actually left. If the core pipeline is not yet returning cited verdicts, that is the only work that matters until it does. **TBD** — confirm current build state before adopting this schedule.

When	Objective	Work	Done when
17 Sep after mentoring	Capture feedback; freeze scope	Rank mentor objections by how often they were raised. Fix the top three only. Everything else becomes a roadmap item. Confirm which P0 items are already working	Written objection list, ranked; scope frozen in writing
18 Sep priority one	Close every P0 gap in the core pipeline	Sid: grounding gate enforced at generation; rule engine complete over Navya's rules. Pratham: all seven endpoints; evidence resolver; deploy to production hosting. Navya: rulebook final; letters collected. Dhiraj: ground-truth set running	Three real cases return cited verdicts on the deployed URL
18 Sep priority two	Appeal generation working end to end	Sid: appeal generator producing a downloadable PDF from established grounds. Parnika: verdict and evidence-trail screens implemented	A Strong verdict yields a downloadable letter without developer intervention
19 Sep morning	Hindi output and Flow C	Pratham: Bhashini integration with offline fallback; Devanagari verified in the PDF download path. Sid: request-for-grounds path. Dhiraj: full regression run, defects logged	Verdict and letter render correctly in Hindi, on screen and in the file
19 Sep afternoon	Harden and rehearse	Feature freeze at noon. Pre-cache the three demo cases. Record the backup run against the final build. Four full rehearsals with a timer. Warm the API. Deploy the final build	The demo runs twice, start to finish, with no developer touching anything
20 Sep	Grand Final	No code changes. None. The build is whatever it was on 19 September	—

If time runs short. Cut in this order: languages beyond the toggle, then Flow C, then Hindi, then the second appeal document. Never cut the grounding gate, the evidence trail, or the Weak-verdict case — those three are what the project is. **Rec**

15. Technical Risks

Risk	Severity	Probability	Mitigation
Core pipeline not yet returning cited verdicts with three days left	Critical	Unknown TBD	Establish actual build state today. If not working, everything except the P0 pipeline stops until it is
Model asserts a clause absent from the policy during the live demo	Critical	Low	Quote-or-abstain enforced at generation, not checked afterwards. Demo cases pre-cached. Never run an uncached case on stage
Too few real rejection letters to test or demonstrate on	Critical	Medium	Collect continuously. Constructed letters are acceptable for testing only, never for accuracy claims and never presented as real
Page references lost in parsing, breaking the evidence trail	High	Medium	Assert page presence at every pipeline boundary. A chunk without a page number fails the build
Devanagari renders as empty boxes in the generated PDF	High	Medium	Embed Noto Sans Devanagari explicitly. Test the download path, not just the screen. Default PDF fonts lack these glyphs
Model provider outage or rate limit on demo day	High	Medium	Second provider configured from the start. Demo path served from cache. Offline translation fallback
Render free-tier cold start makes the demo look broken	Medium	High	Warm the API before the session. Cached demo cases bypass the cold path entirely
Insurer letter formats too varied for reliable extraction	Medium	Medium	Cover more than one insurer in the labelled set. Flow C handles the degraded case as a feature
Scanned image-only policy PDFs cannot be page-cited	Medium	Medium	Detect and reject with a clear message rather than parsing unreliably
The 2020 non-payable list turns out to be repealed	Medium	Medium	Resolve before coding those rules. If unresolved, ship without that rule set rather than on an uncertain basis
Integration failure between ingestion and reasoning	Medium	Medium	Sid and Pratham agree the claim-record contract in writing before either builds against it
A dependency upgrade breaks the build near the deadline	Medium	Low	Pin every version. No dependency changes after 18 September

16. Estimated Infrastructure Requirements

Resource	Requirement	Provider	Cost
Frontend hosting	Static site, single region	Vercel free tier	None for the MVP
Backend compute	One small container. Single concurrent user for the MVP	Render free tier	None for the MVP. Paid tier needed to remove cold starts TBD
Database	Postgres with pgvector. Well under free-tier limits at demo volume	Supabase free tier	None for the MVP
Object storage	Session-scoped only. Documents deleted at session end, so storage does not accumulate	Supabase Storage	None for the MVP
Model inference	Dominated by long-context calls over a 40-80 page policy. Several calls per analysis	Two LLM providers	TBD — depends on provider and token volume; measure per-analysis cost during the build

Resource	Requirement	Provider	Cost
Translation	Verdict and letter only, on request	Bhashini; IndicTrans2 local fallback	None expected
CI	Test run on push	GitHub Actions free tier	None for the MVP
Fonts	Noto Sans Devanagari for PDF generation	Bundled in the repository	None. Open licence

No cost figures are estimated. Per-analysis inference cost is the only material variable and it has not been measured. Instrument token usage during the build so the figure is real before it appears in any business conversation. **TBD**

17. Cross-Document Consistency Check

The three documents were checked against each other and against the submission deck. Problem statement, target users, MVP scope, core features, business model and team allocation agree across all three. Four items do not resolve themselves and need a team decision.

Consistency Issues Requiring Team Decision

#	Issue	The conflict	Decision needed
1	The timeline has already elapsed	The deck schedules 14–16 September for the core pipeline and 17 September for mentoring. Today is 17 September. This technology document rewrites the plan against three remaining days; the deck and the PRD still carry the original dates	Confirm the actual build state today, then adopt §14 as the operative plan and treat the deck's timeline as historical
2	Appeal pack scope differs between deck and PRD	The deck's output slide shows a full appeal pack — grievance-officer letter, portal complaint text and Ombudsman form. The PRD makes only the grievance-officer letter a P0 and moves the other two to P2	Either accept the PRD's narrower MVP and describe the full pack as roadmap when presenting, or raise the other two documents to P0 and find the time
3	The escalation clock's status	The deck shows the escalation clock on the output slide as a delivered feature. The PRD places it at P2, outside the MVP	Decide whether to build a simple static version for the demo or to present it explicitly as Phase 2
4	The 2020 non-payable items circular	The deck and the PRD both rely on the 146 standardised non-payable items. Whether that 2020 circular survived the repeal schedule of the 2024 Master Circular has not been confirmed	Resolve before coding those rules. If unresolved by 18 September, ship the rule engine without that rule set

Verified as consistent. The problem statement, the five verified statistics, target users and stakeholders, the non-goals list, the free-for-individuals business model, the secondary-customer segments, the technology stack, the team allocation, and the grounding contract as the stated innovation all agree across the deck, the PRD, the BRD and this document.

Document map

Document	Defines	Use it when
Pratkar PRD	What we are building	Settling whether something is in scope; writing acceptance criteria; deciding what to cut
Pratkar BRD	Why it should exist	Answering mentors and judges on market, model and competitive position
This document	How it is built	Making an implementation decision; onboarding a contributor; assigning work